RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Department of Computer Science and Engineering

CS23334 Fundamentals of Data Science Lab

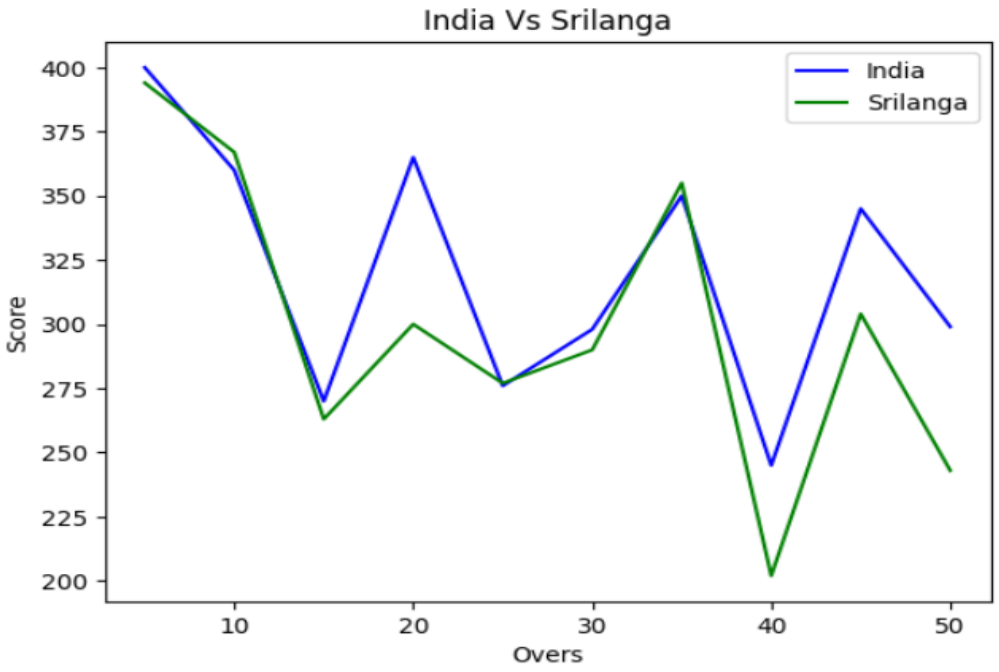
III semester II Year (2023R)

Name of the Student : SWETA.S.K

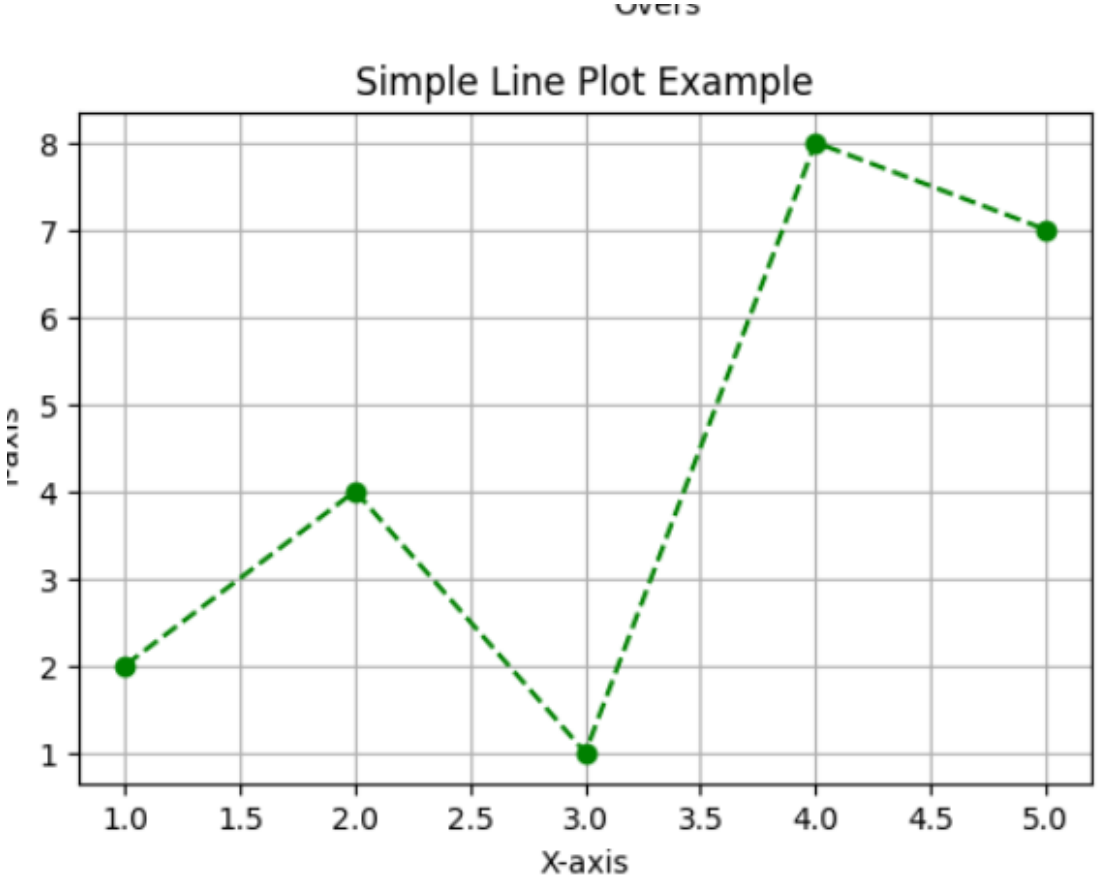
Register Number : 240701549

EX 1

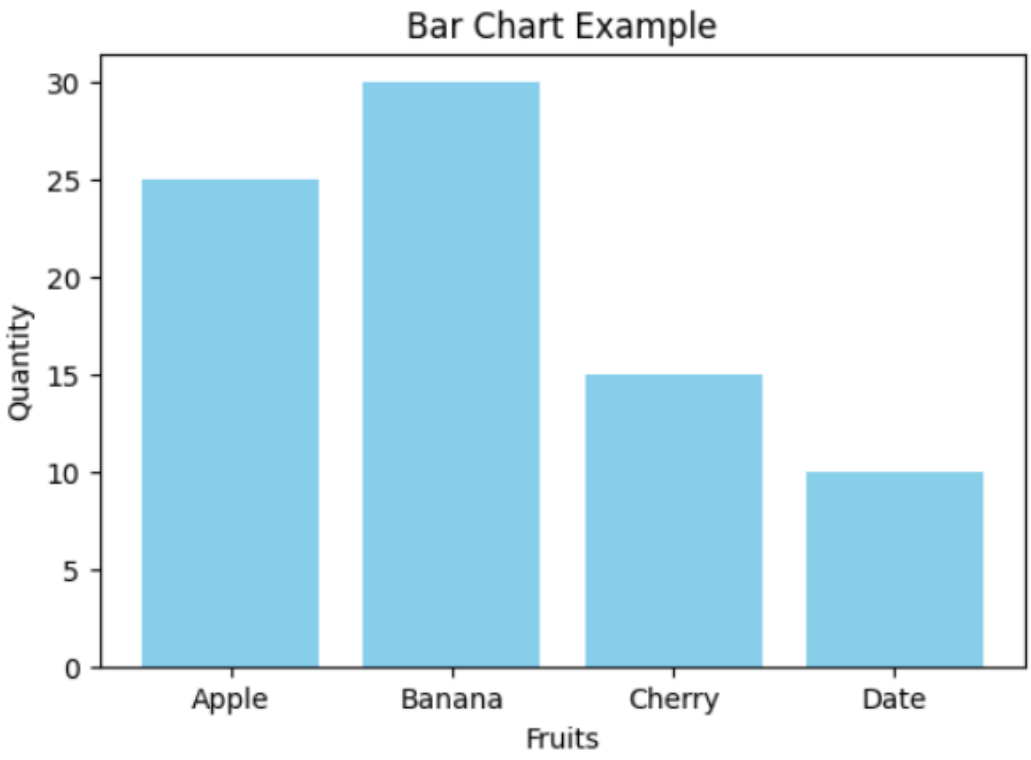
```
#Sweta.S.K
#240701549

import matplotlib.pyplot as plt
import numpy as np

# --- LINE PLOT ---
# Corrected code for the "India Vs Srilanga" plot
overs = list(range(5, 51, 5))
India_Score = [400, 360, 270, 365, 276, 298, 350, 245, 345, 299]
Srilanga_Score = [394, 367, 263, 300, 277, 290, 355, 202, 304, 243]

# Set up the plot details
plt.plot(overs, India_Score, color="blue", label="India")
plt.plot(overs, Srilanga_Score, color="green", label="Srilanga")
plt.title("India Vs Srilanga")
plt.xlabel("Overs")
plt.ylabel("Score")
plt.legend()
```



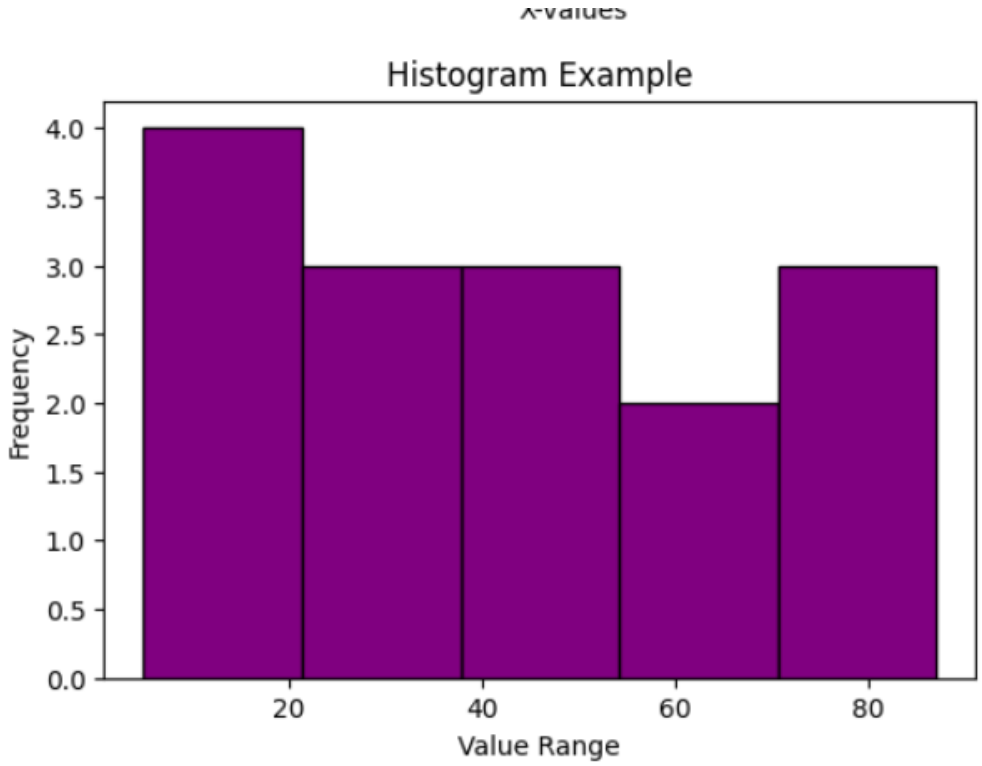
```python
#Sweta.S.K
#240701549

import matplotlib.pyplot as plt
import numpy as np

# --- SIMPLE LINE PLOT ---
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 8, 7]

plt.figure(figsize=(6, 4)) # Set the figure size
plt.plot(x, y, marker='o', linestyle='--', color='green')
plt.title("Simple Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
```



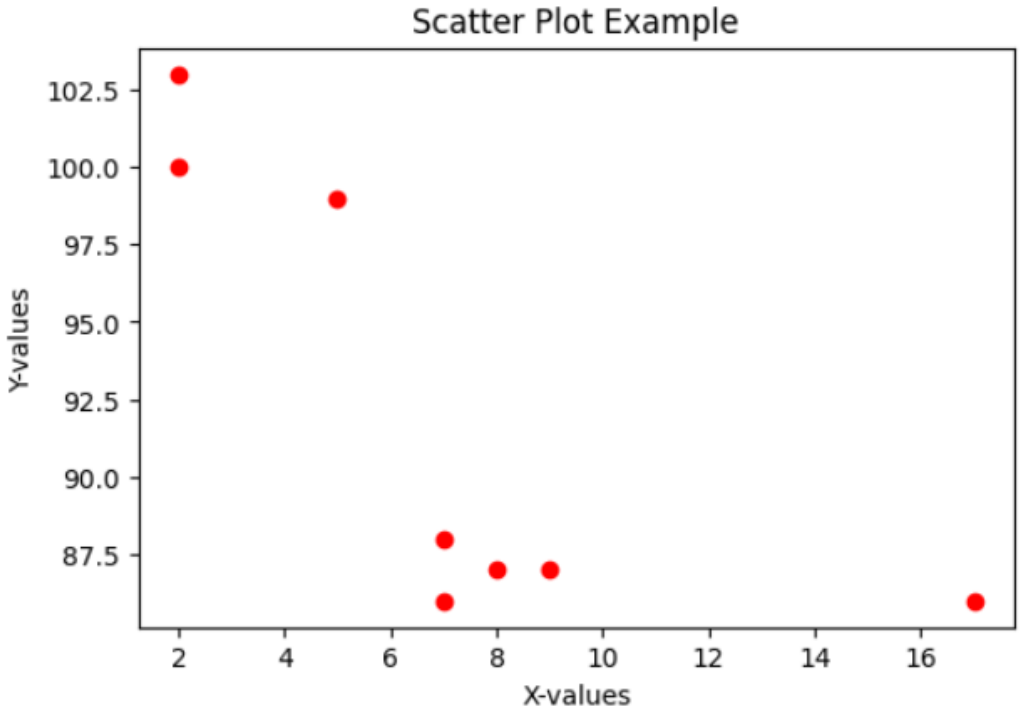
```python
#Sweta.S.K
#240701549

import matplotlib.pyplot as plt
import numpy as np
fruits = ['Apple', 'Banana', 'Cherry', 'Date']
quantity = [25, 30, 15, 10]

plt.figure(figsize=(6, 4))
plt.bar(fruits, quantity, color='skyblue')
plt.title("Bar Chart Example")
plt.xlabel("Fruits")
plt.ylabel("Quantity")
plt.show()
```



```python
#Sweta.S.K
#240701549

import matplotlib.pyplot as plt
import numpy as np
x_scatter = [5, 7, 8, 7, 2, 17, 2, 9]
y_scatter = [99, 86, 87, 88, 100, 86, 103, 87]

plt.figure(figsize=(6, 4))
plt.scatter(x_scatter, y_scatter, color='red')
plt.title("Scatter Plot Example")
plt.xlabel("X-values")
plt.ylabel("Y-values")
plt.show()
```



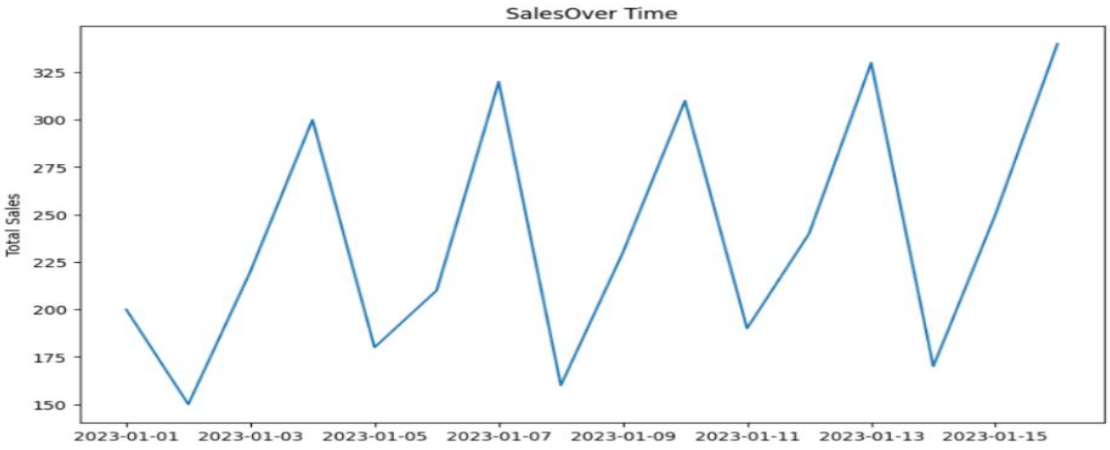
```python
#Sweta.S.K
#240701549

import matplotlib.pyplot as plt
import numpy as np
data = [22, 87, 5, 43, 56, 73, 55, 54, 11, 20, 51, 5, 79, 31, 27]

plt.figure(figsize=(6, 4))
plt.hist(data, bins=5, color='purple', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.show()
```

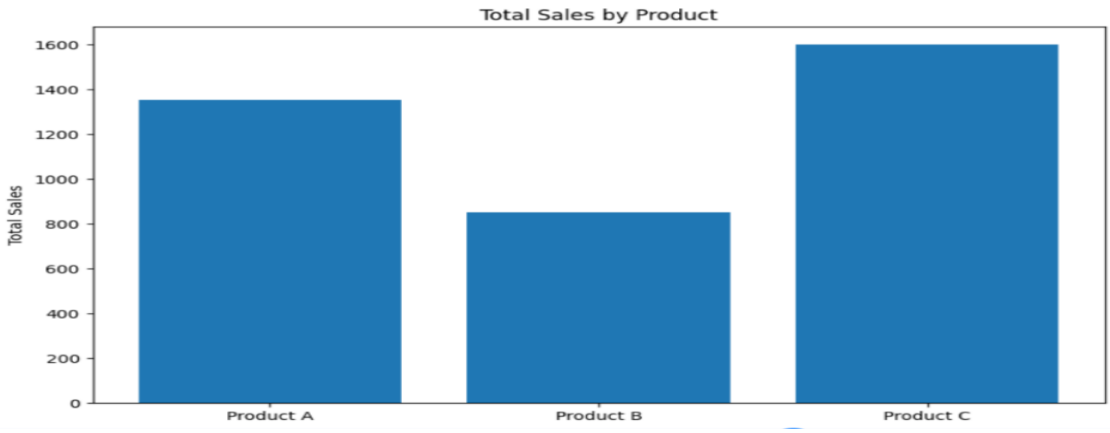


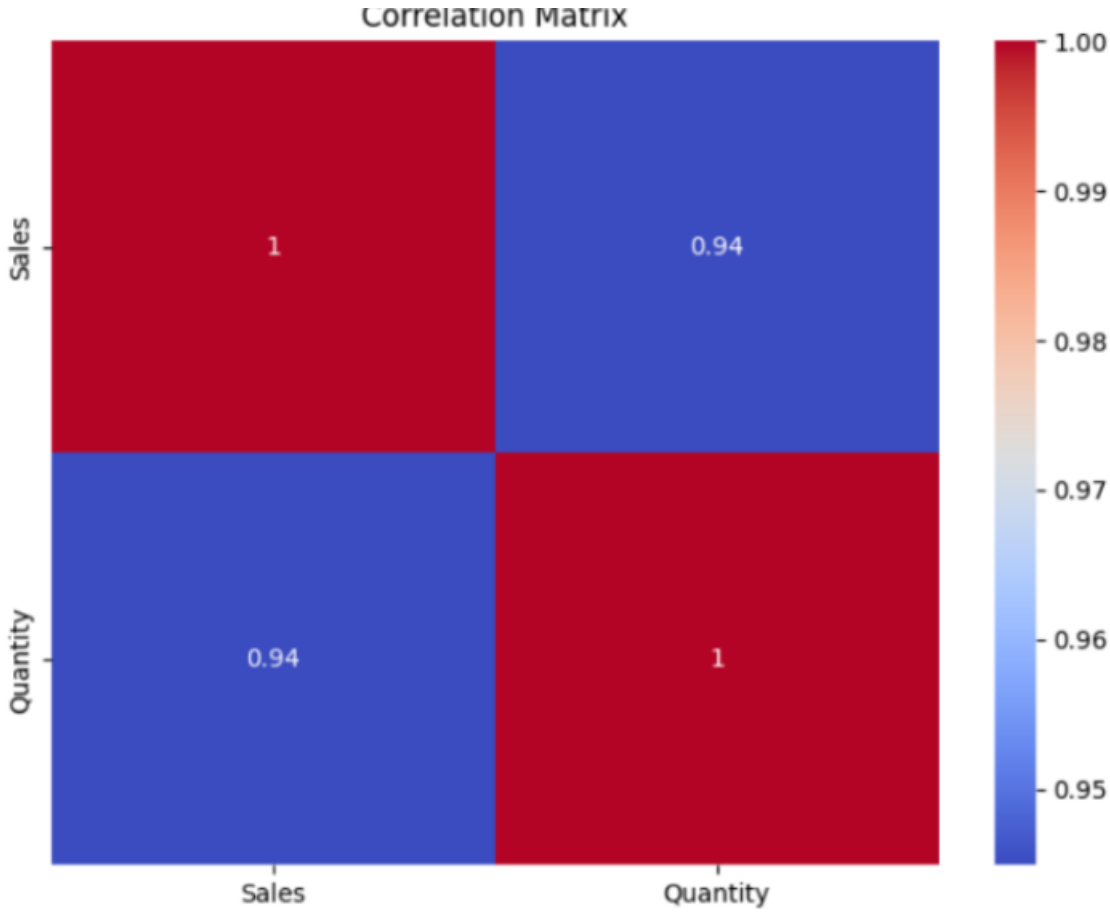
EX 2

```
#Sweta.S.K
#240701549


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

file_path = '/sales_data .csv' # Corrected filename to include space

df = pd.read_csv(file_path)

print("--- Head ---")
print(df.head())

print("\n--- Missing Values ---")
print(df.isnull().sum())

df['Sales'].fillna(df['Sales'].mean(), inplace=True);
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print("\n--- Description ---")
print(df.describe())

product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
        'Quantity': 'sum'
        }).reset_index()

print("\n--- Product Summary ---")
print(product_summary)

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'], marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.grid(True)
plt.show()
```

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```


SalesOver Time
Total Sales
325
300
275
250
225
200
175
150
2023-01-01
2023-01-03
2023-01-05
2023-01-07
2023-01-09
2023-01-11
2023-01-13
2023-01-15


Total Sales by Product
Total Sales
1600
1400
1200
1000
800
600
400
200
0
Product A
Product B
Product C


Correlation Matrix
Sales
Quantity
1
0.94
0.94
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

EX 3

```
#Sweta.S.K
#240701549


import numpy as np
import pandas as pd

file_path = '/content/pre-process_datasample.csv'

df = pd.read_csv(file_path)
print("--- Original DataFrame ---")
print(df)
print("\n")
df.info()

country_dummies = pd.get_dummies(df.Country)
print("\n--- One-Hot Encoded Countries ---")
print(country_dummies)

updated_dataset = pd.concat([country_dummies, df.iloc[:, 1:]], axis=1)
print("\n--- Concatenated DataFrame ---")
print(updated_dataset)

updated_dataset['Purchased'].replace(['No', 'Yes'], [0, 1], inplace=True)
print("\n--- Final DataFrame with Encoded Purchased ---")
print(updated_dataset)
print("\n")
updated_dataset.info()
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	NaN	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes
```

d

	France	Germany	Spain	Age	Salary	Purchased
0	1	0	0	44.0	72000.0	No
1	0	0	1	27.0	48000.0	Yes
2	0	1	0	30.0	54000.0	No
3	0	0	1	38.0	61000.0	No
4	0	1	0	40.0	63778.0	Yes
5	1	0	0	35.0	58000.0	Yes
6	0	0	1	38.0	52000.0	No
7	1	0	0	48.0	79000.0	Yes
8	1	0	0	50.0	83000.0	No
9	1	0	0	37.0	67000.0	Yes

	France	Germany	Spain
0	1	0	0
1	0	0	1
2	0	1	0
3	0	0	1
4	0	1	0
5	1	0	0
6	0	0	1
7	1	0	0
8	1	0	0
9	1	0	0

EX 4

```
#Sweta.S.K
#240701549


import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")
print("--- Original DataFrame ---")
print(df)

df.drop_duplicates(inplace=True)
print("\n--- After Dropping Duplicates ---")
print(df)

df.drop('Age_Group.1', axis=1, inplace=True)
print("\n--- After Dropping Extra Column ---")
print(df.head())

df.loc[df.CustomerID < 0, 'CustomerID'] = np.nan
df.loc[df.Bill < 0, 'Bill'] = np.nan
df.loc[df['Rating(1-5)'] < 1, 'Rating(1-5)'] = np.nan
df.loc[df['Rating(1-5)'] > 5, 'Rating(1-5)'] = np.nan
df.loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20), 'NoOfPax'] = np.nan
df.loc[df.EstimatedSalary < 0, 'EstimatedSalary'] = np.nan

print("\n--- After Replacing Inappropriate Data with NaN ---")
print(df)
print("\n--- Missing Values Count ---")
print(df.isnull().sum())

df['EstimatedSalary'].fillna(round(df['EstimatedSalary'].mean()), inplace=True)
df['NoOfPax'].fillna(round(df['NoOfPax'].median()), inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df['Bill'].fillna(round(df['Bill'].mean()), inplace=True)
df.dropna(subset=['CustomerID'], inplace=True)


print("\n--- After Filling Missing Data ---")
print(df)
print("\n--- Final Missing Values Count ---")
print(df.isnull().sum())
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSala
0	1	20-25	4	Ibis	veg	1300	2	4000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	5900
2	3	25-30	6	RedFox	Veg	1322	2	3000
3	4	20-25	-1	LemonTree	Veg	1234	2	12000
4	5	35+	3	Ibis	Vegetarian	989	2	4500
5	6	35+	3	Ibys	Non-Veg	1909	2	12222
6	7	35+	4	RedFox	Vegetarian	1000	-1	2112
7	8	20-25	7	LemonTree	Veg	2999	-10	34567
8	9	25-30	2	Ibis	Non-Veg	3456	3	-9999
9	9	25-30	2	Ibis	Non-Veg	3456	3	-9999
10	10	30-35	5	RedFox	non-Veg	-6755	4	8777

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1.0	20-25	4	Ibis	Veg	1300.0	2.0	40000.0
1	2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
2	3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
3	4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
4	5.0	35+	3	Ibis	Veg	989.0	2.0	45000.0
5	6.0	35+	3	Ibis	Non-Veg	1909.0	2.0	122220.0
6	7.0	35+	4	RedFox	Veg	1000.0	2.0	21122.0
7	8.0	20-25	7	LemonTree	Veg	2999.0	2.0	345673.0
8	9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	96755.0
9	10.0	30-35	5	RedFox	Non-Veg	1801.0	4.0	87777.0

EX 5

```python
#Sweta.S.K
#240701549
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
array = np.random.randint(1, 100, 16)
print("Original array:", array)

def outDetection(data):
    sorted(data)
        Q1, Q3 = np.percentile(data, [25, 75])
            IQR = Q3 - Q1
                lr = Q1 - (1.5 * IQR)
                    ur = Q3 + (1.5 * IQR)
                        return lr, ur

                        lr, ur = outDetection(array)
                        print("Lower range:", lr)
                        print("Upper range:", ur)

                        sns.histplot(array, kde=True)
                        plt.title("Original Data Distribution")
                        plt.show()

                        new_array = array[(array > lr) & (array < ur)]
                        print("Array after filtering outliers:", new_array)

                        sns.histplot(new_array, kde=True)
                        plt.title("Filtered Data Distribution")
                        plt.show()
```

EX 6

```python
#Sweta.S.K
#240701549
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

df = pd.read_csv('/content/pre-process_datasample.csv')
print("--- Original DataFrame ---")
print(df)

df.dropna(subset=['Country'], inplace=True)
print("\n--- DataFrame after dropping NaN Country ---")
print(df)

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
features[:, 1:3] = imputer.fit_transform(features[:, 1:3])
print("\n--- Features after Imputing Age and Salary ---")
print(features)

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='passthrough')
features = np.array(ct.fit_transform(features))
print("\n--- Features after One-Hot Encoding Country ---")
print(features)

le = LabelEncoder()
label = le.fit_transform(label)
print("\n--- Encoded Label (Purchased) ---")
print(label)

sc = StandardScaler()
features = sc.fit_transform(features)
print("\n--- Scaled Features ---")
print(features)
```

EX 7

```
#Sweta.S.K
#240701549
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')
print("--- Tips Dataset Head ---")
print(tips.head())

sns.displot(tips['total_bill'], kde=True)
plt.title("Distribution of Total Bill")
plt.show()

sns.jointplot(x='total_bill', y='tip', data=tips, kind='scatter')
plt.show()

sns.pairplot(tips, hue='sex')
plt.show()

sns.rugplot(tips['total_bill'])
plt.show()

sns.heatmap(tips.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

sns.boxplot(x='day', y='total_bill', data=tips, hue='smoker')
plt.title("Box Plot of Total Bill by Day and Smoker")
plt.show()

sns.countplot(x='day', data=tips, hue='sex')
plt.title("Count of Visits by Day and Sex")
plt.show()
```

How ca

EX 8

```
#Sweta.S.K
#240701549
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

df = pd.read_csv('Salary_data.csv')
print("--- Salary Data Head ---")
print(df.head())
print("\n")
df.info()

features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values

x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)

print(f"\nTraining Score (R-squared): {model.score(x_train, y_train)}")
print(f"Test Score (R-squared): {model.score(x_test, y_test)}")

y_pred = model.predict(x_test)

plt.scatter(x_train, y_train, color='blue', label='Actual Salaries')
plt.plot(x_train, model.predict(x_train), color='red', label='Regression Line')
plt.title('Salary vs Experience (Training Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()

plt.scatter(x_test, y_test, color='green', label='Actual Salaries')
plt.plot(x_test, y_pred, color='red', label='Regression Line')
plt.title('Salary vs Experience (Test Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```

EX 9

```python
#Sweta.S.K
#240701549
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv('Social_Network_Ads.csv')
print("--- Social Network Ads Head ---")
print(df.head())

features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values

x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=42)

sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)

print(f"\nTraining Score (Accuracy): {finalModel.score(x_train, y_train)}")
print(f"Test Score (Accuracy): {finalModel.score(x_test, y_test)}")

y_pred = finalModel.predict(x_test)

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))

print(f"\nTest Accuracy: {accuracy_score(y_test, y_pred)}")
```

EX 10

```python
#Sweta.S.K
#240701549
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('Iris.csv')
print("--- Iris Data Head ---")
print(df.head())
print("\n")
df.info()
print("\n--- Variety Value Counts ---")
print(df.variety.value_counts())

features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

xtrain, xtest, ytrain, ytest = train_test_split(features, label, test_size=0.2, random_state=42)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)

print(f"\nTraining Score (Accuracy): {model_KNN.score(xtrain, ytrain)}")
print(f"Test Score (Accuracy): {model_KNN.score(xtest, ytest)}")

all_predictions = model_KNN.predict(features)

print("\n--- Confusion Matrix (on all data) ---")
print(confusion_matrix(label, all_predictions))

print("\n--- Classification Report (on all data) ---")
print(classification_report(label, all_predictions))

test_predictions = model_KNN.predict(xtest)
print("\n--- Classification Report (on TEST data) ---")
print(classification_report(ytest, test_predictions))
```

EX 11

```
#Sweta.S.K
#240701549
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv('Mall_Customers.csv')

df.info()

df.head()

features = df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=0)
        kmeans.fit(features)
            wcss.append(kmeans.inertia_)

            plt.plot(range(1,11), wcss)
            plt.title('Elbow Method')
            plt.xlabel('No. of Clusters')
            plt.ylabel('WCSS')
            plt.show()
```

How can I install Python libraries?

EX 12

```python
#Sweta.S.K
#240701549
import numpy as np
from scipy import stats

marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70

t_stat, p_value = stats.ttest_1samp(marks, mu_0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis -> Mean is significantly different from 70.")
    else:
        print("Fail to Reject Null Hypothesis -> No significant difference.")
```

EX 13

```python
#Sweta.S.K
#240701549
from math import sqrt
from scipy.stats import norm

x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36

z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
p_value = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis -> Mean is significantly different from 50 g.")
    else:
        print("Fail to Reject Null Hypothesis No significant difference.")
```

EX 14

```python
#Sweta.S.K
#240701549
import numpy as np
from scipy import stats

A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis -> Significant difference among fertilizers.")
    else:
        print("Fail to Reject Null Hypothesis -> No significant difference.")
```